



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

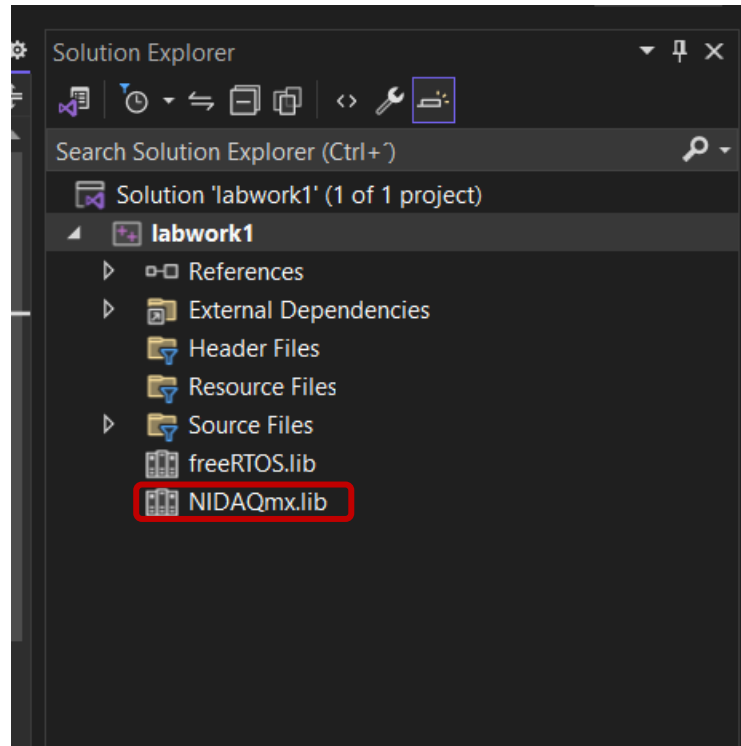
LAB #1 – class 1/5

INTERACTION WITH THE “PHYSICAL” SYSTEM

2025 - 2026

Interaction with physical system

- ☐ Using the Digital Acquisition board (DAQ)
- ☐ Introducing the kit “Splitter Conveyor”
- ☐ The physical vs simulated Splitter Conveyor
- ☐ Reading sensors & using the actuators



NIDAQmx.lib, which is the library for letting your application interact with the Splitter Conveyor kit, through an acquisition board...

The NI USB-6509 DAQ Board

□ A few features

- Data acquisition (DAQ).
- The NI USB-6509 features 96 TTL/CMOS-compatible digital I/O lines,
- Watchdog timer, digital input filtering and change detection...
- Output programmable power-up states.
- ...



USB connects in your laptop

Connects to the physical Splitter Conveyor kit (or other one)

DIGITAL ACQUISITION BOARD



The NI USB-6509 DAQ Board



Figure 6. Pin Assignments for the NI USB-6509 using the SH100-100-F

Pin assignment:

P2.7	1	51	P8.7
P5.7	2	52	P11.7
P2.6	3	53	P8.6
P5.6	4	54	P11.6
P2.5	5	55	P8.5
P5.5	6	56	P11.5
P2.4	7	57	P8.4
P5.4	8	58	P11.4
P2.3	9	59	P8.3
P5.3	10	60	P11.3
P2.2	11	61	P8.2
P5.2	12	62	P11.2
P2.1	13	63	P8.1
P5.1	14	64	P11.1
P2.0	15	65	P8.0
P5.0	16	66	P11.0
P4.7	17	67	P7.7
P1.7	18	68	P10.7
P1.6	19	69	P7.6
P4.6	20	70	P10.6
P1.5	21	71	P7.5
P4.5	22	72	P10.5
P1.4	23	73	P7.4
P4.4	24	74	P10.4
P1.3	25	75	P7.3
P4.3	26	76	P10.3
P1.2	27	77	P7.2
P4.2	28	78	P10.2
P1.1	29	79	P7.1
P4.1	30	80	P10.1
P1.0	31	81	P7.0
P4.0	32	82	P10.0
P0.7	33	83	P6.7
P0.6	34	84	P6.6
P0.5	35	85	P6.5
P0.4	36	86	P6.4
P0.3	37	87	P6.3
P0.2	38	88	P6.2
P0.1	39	89	P6.1
P0.0	40	90	P6.0
P3.3	41	91	P9.3
P3.2	42	92	P9.2
P3.1	43	93	P9.1
P3.0	44	94	P9.0
P3.0	45	95	P8.1
P3.1	46	96	P8.0
P3.0	47	97	P8.0
P3.0	48	98	P8.0
+5 V	49	99	+5 V
GND	50	100	GND

The NI USB-6509 DAQ Board

- ❑ Left side lies the 96 DIO, which can be set programmatically as input/output inside your C++ code.
- ❑ For Diagnostic and testing, the NI MAX App can be used.

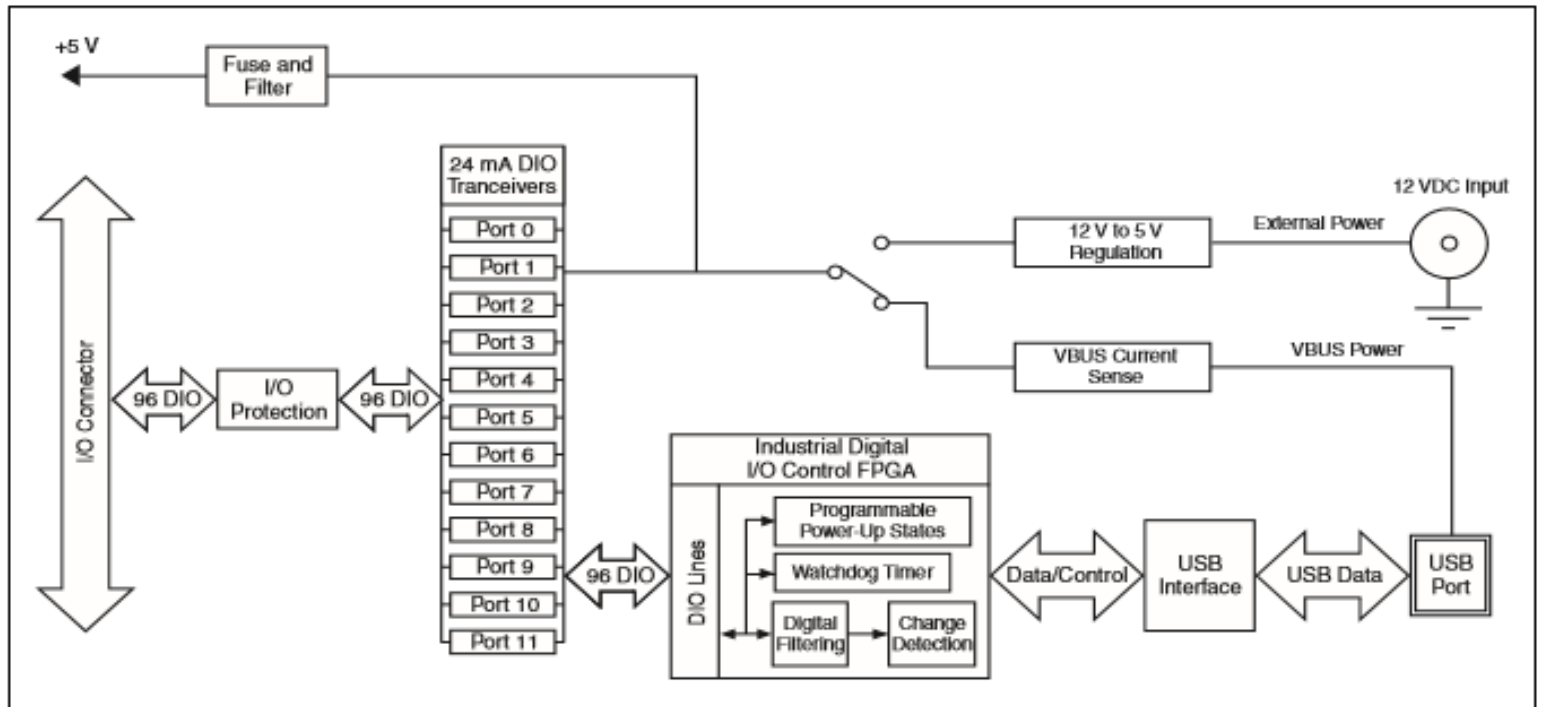
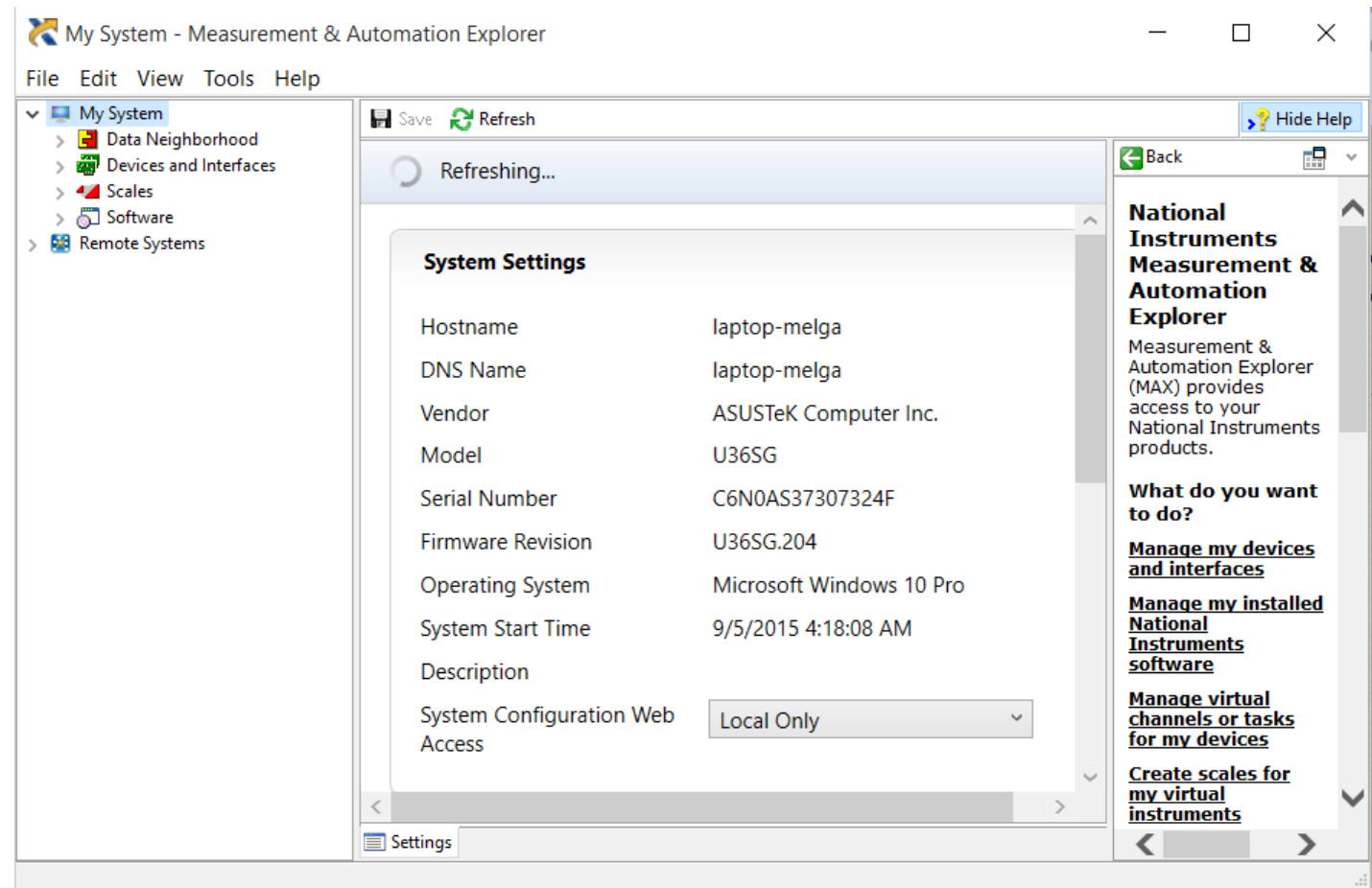


Figure 4. NI USB-6509 Block Diagram

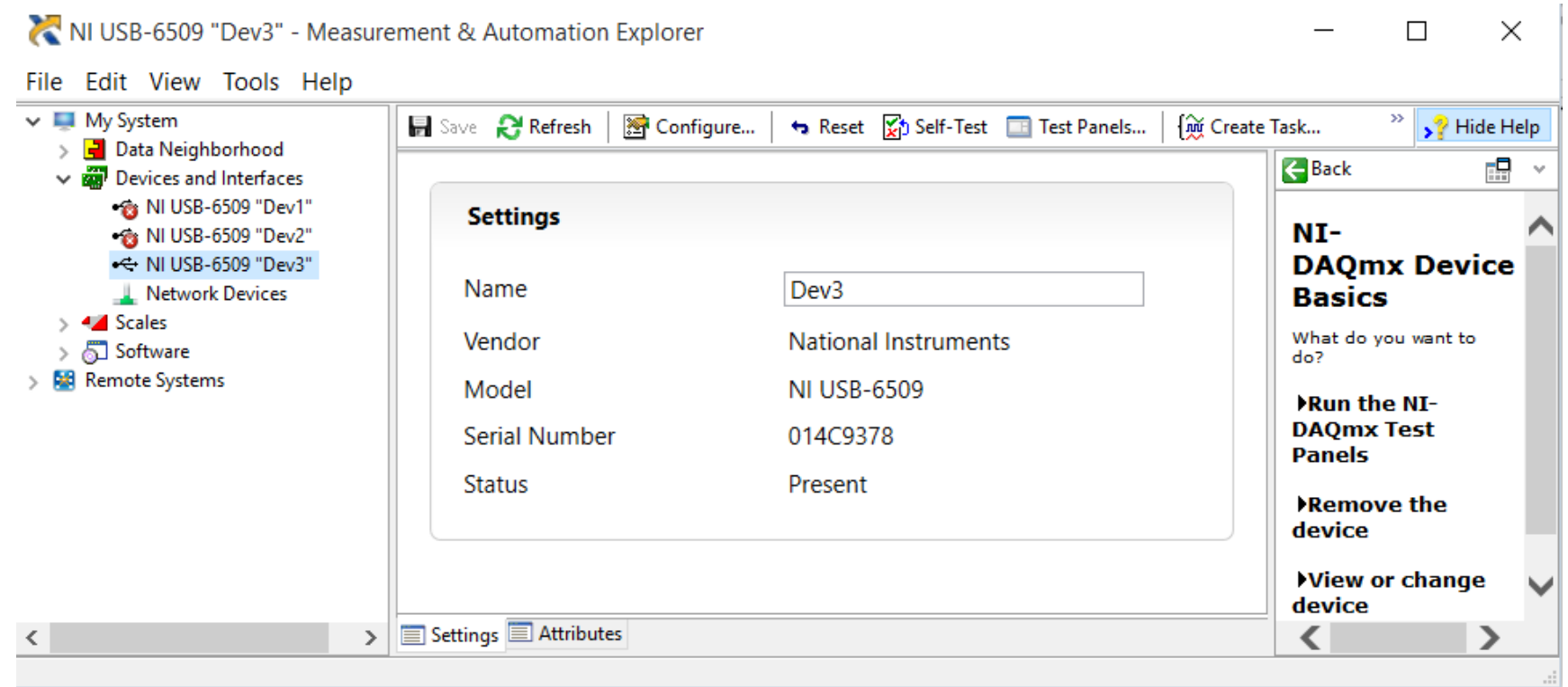
The NI USB-6509 DAQ Board

❑ Device configuration application



The NI USB-6509 DAQ Board

- Device selection

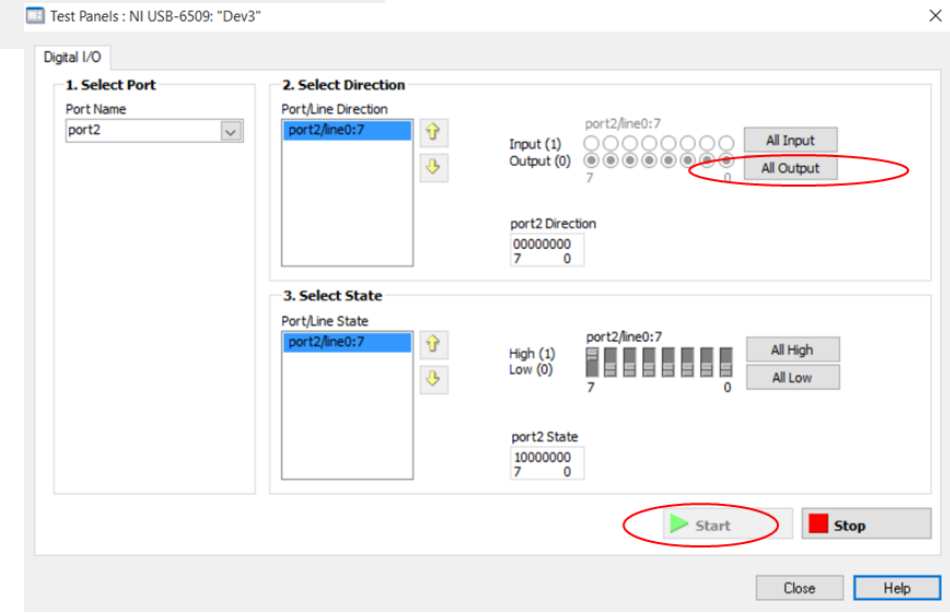
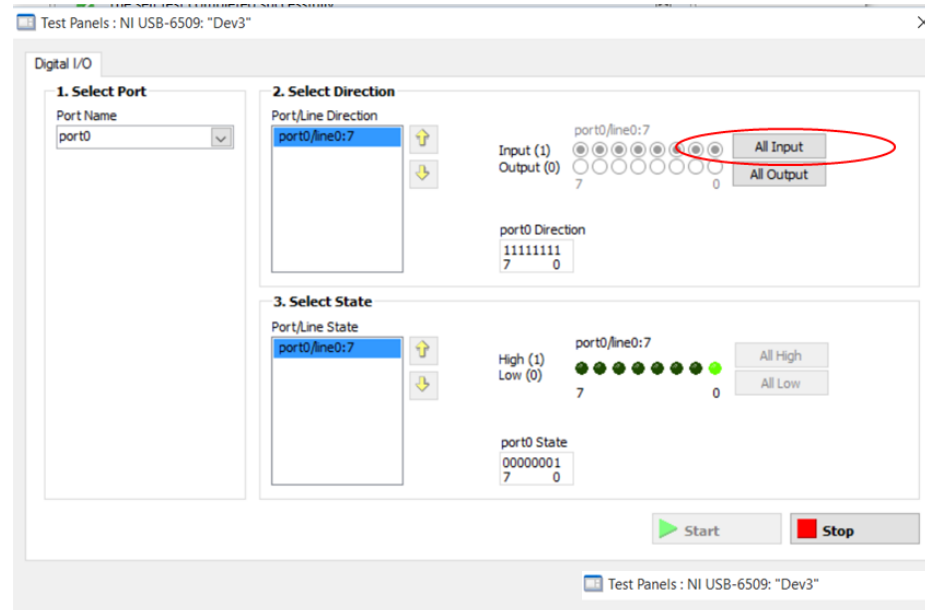


DIGITAL ACQUISITION BOARD

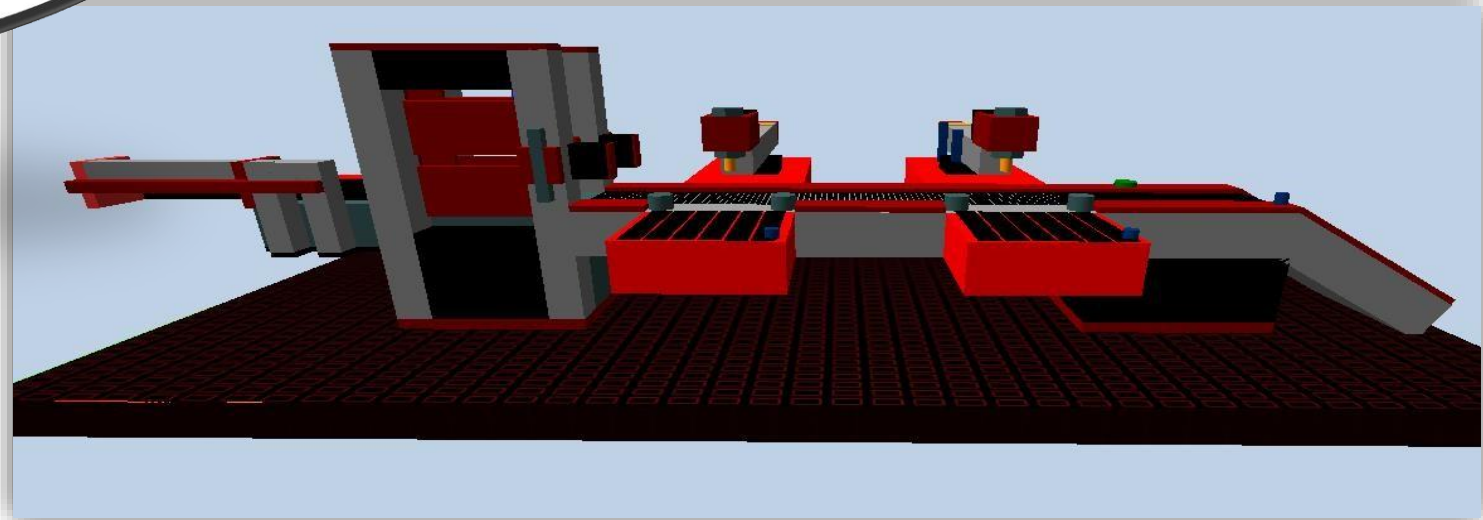
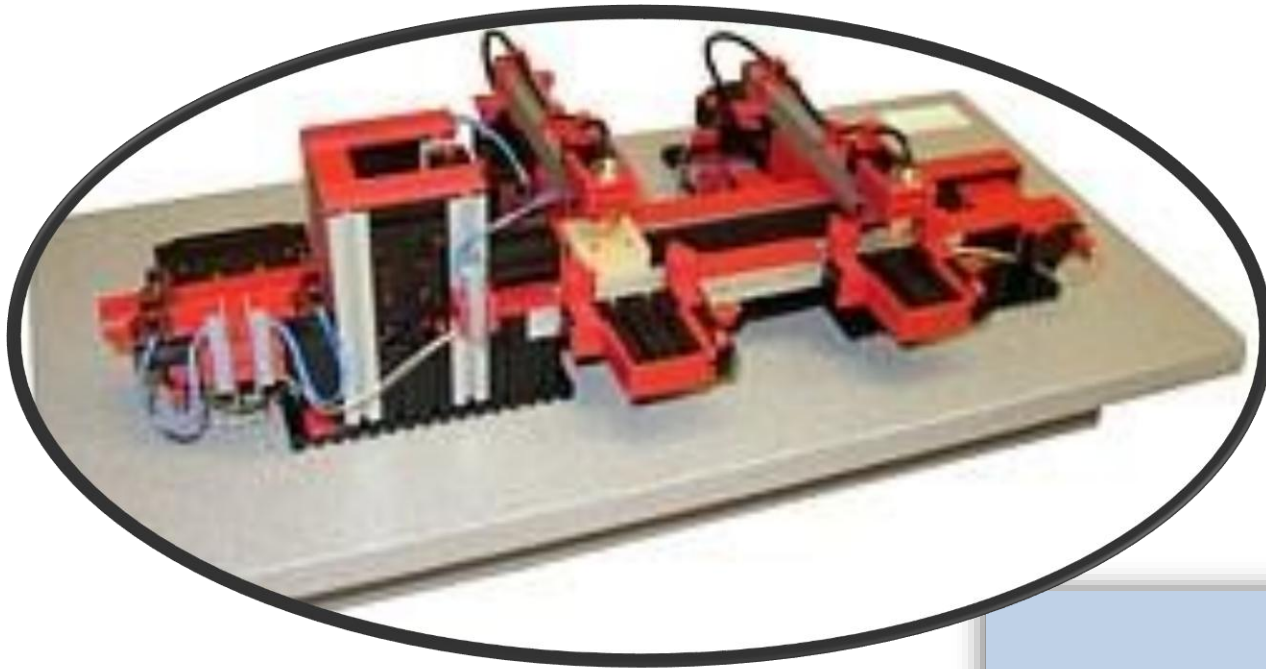


The NI USB-6509 DAQ Board

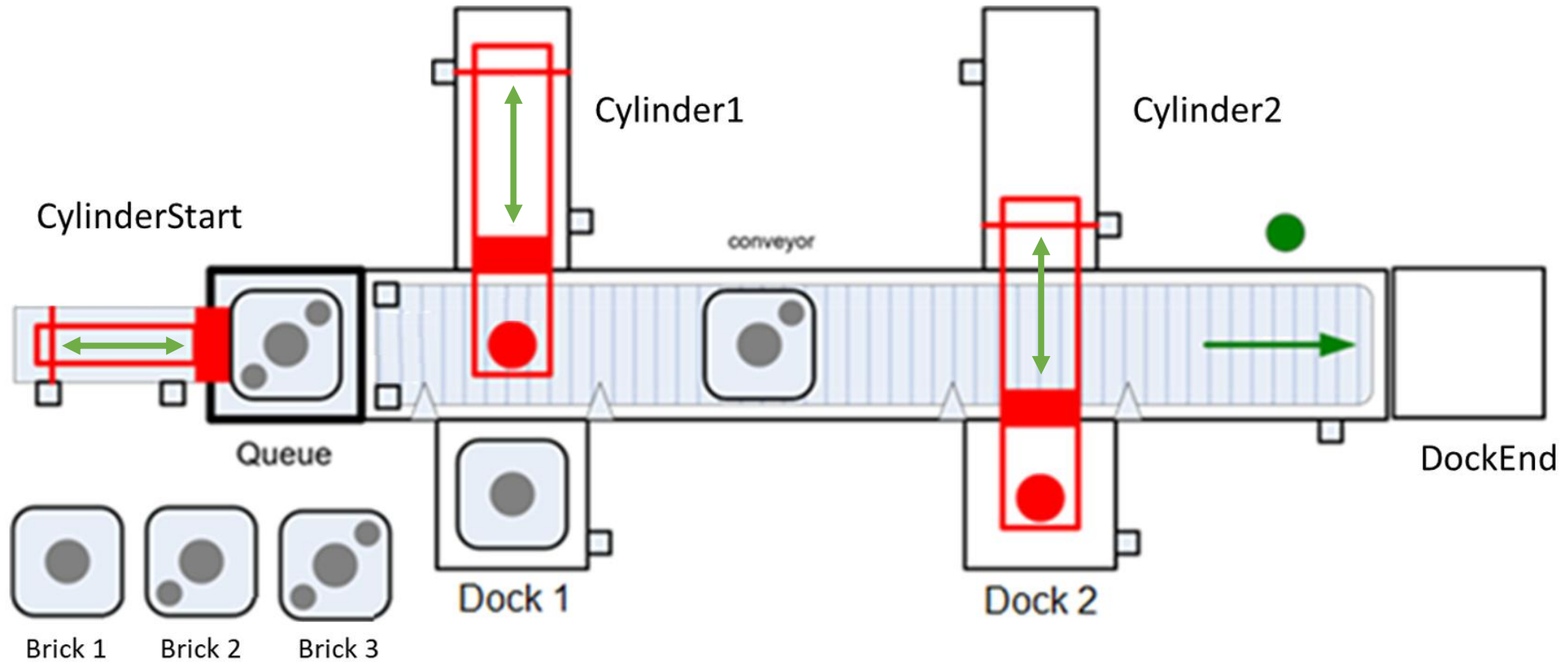
- ☐ Test panels: Port 0 and 1: all inputs
- ☐ set port 0 and 1 bits as inputs
- ☐ set port 2 bits as outputs



INTRODUCING THE KIT PHYSICAL SPLITTER CONVEYOR

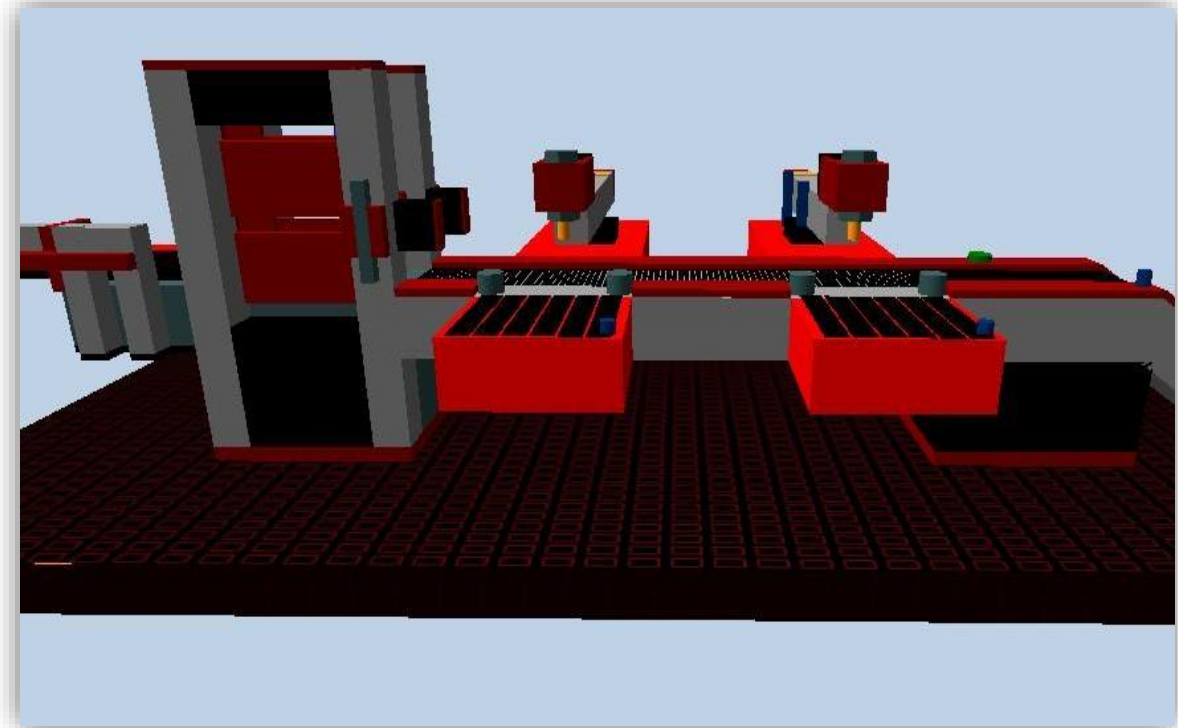


INTRODUCING THE KIT PHYSICAL SPLITTER CONVEYOR



Using the Simulator

- ❑ There is a simulator to replace the physical kit to overcome remote classes context (sometimes, even in classroom, due to the number of students)
- ❑ All functionality and coding errors can be solved using the simulator.
- ❑ Switching from kit to simulator could be done without any changes in your C++ code.



CONFIGURING LABWORK1 PROJECT PROPERTIES



More Project Configuration

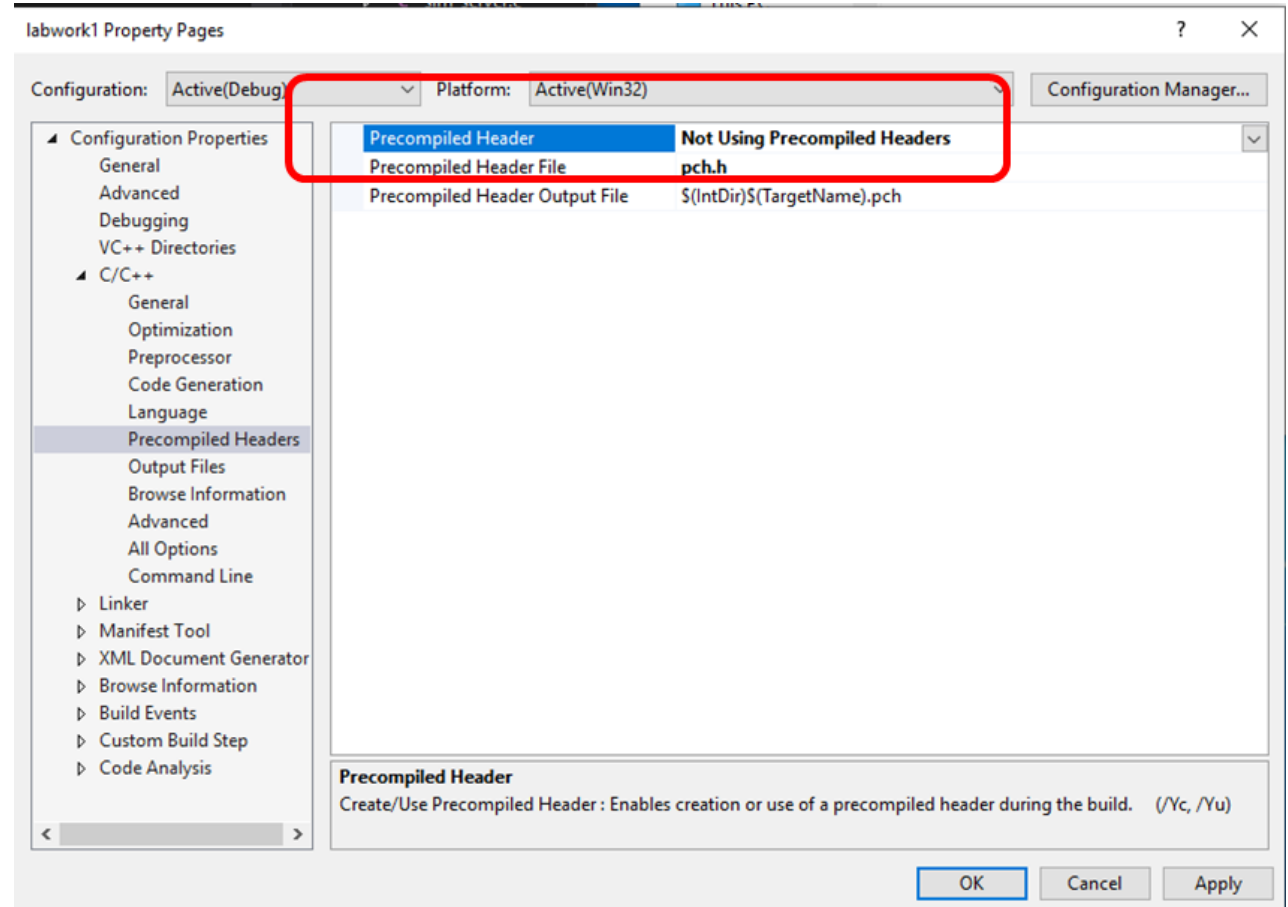
- It tells the compiler to look for *.h include files in its own project folder

LabWork1 Property Pages

Configuration:	Active(Debug)	Platform:	Active(Win32)
Configuration Properties			
General			
Debugging			
VC++ Directories			
C/C++			
General			
Optimization			
Preprocessor			
Code Generation			
Language			
Precompiled Headers			
Output Files			
Browse Information			
Additional Include Directories			;c:\str2018\include;c:\s
Additional #using Directories			
Debug Information Format			Program Database for Executable Files (/ZI)
Support Just My Code Debugging			Yes (/JMC)
Common Language RunTime Support			
Consume Windows Runtime Extension			
Suppress Startup Banner			Yes (/nologo)
Warning Level			Level3 (/W3)
Treat Warnings As Errors			No (/WX-)
Warning Version			
Diagnostics Format			Classic (/diagnostics:classic)
SDL checks			Yes (/sdl)

More Project Configuration

- Change pre-compiled headers option
- Select “Not Using Precompiled Headers”
- Using it would be important in big projects (with many modules)



CONFIGURING LABWORK1 PROJECT PROPERTIES



More Project Configuration

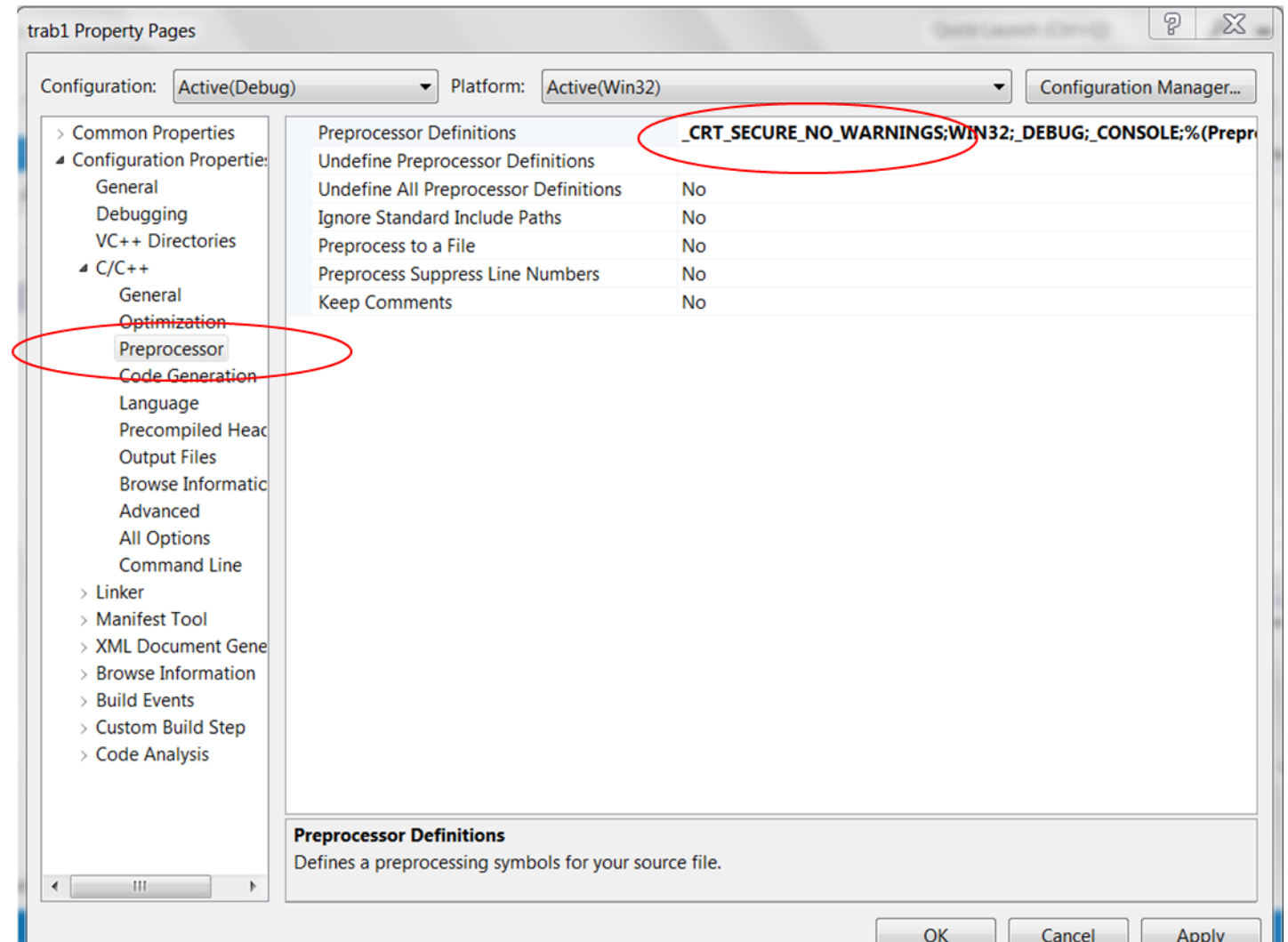
- Configure (add)

“_CRT_SECURE_NO_WARNINGS”

(in Project properties)

Copy past:

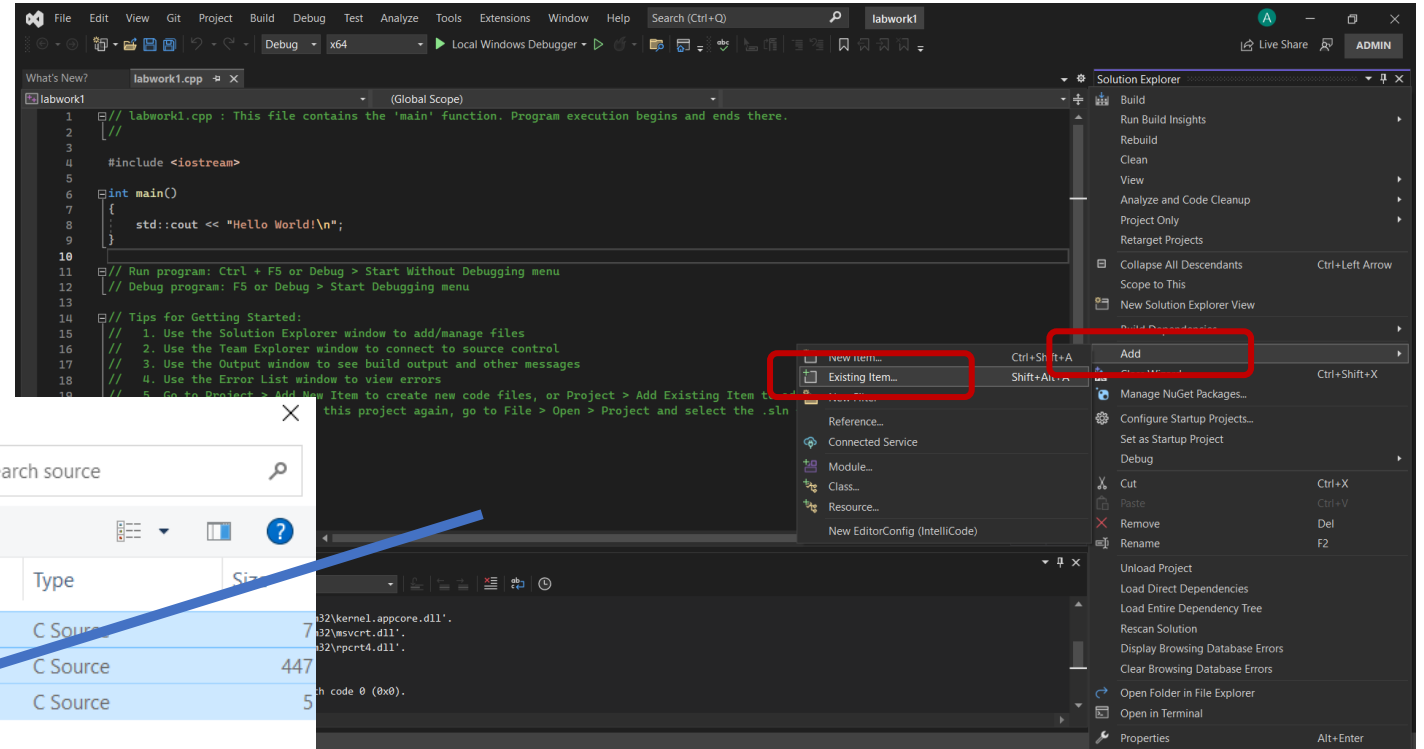
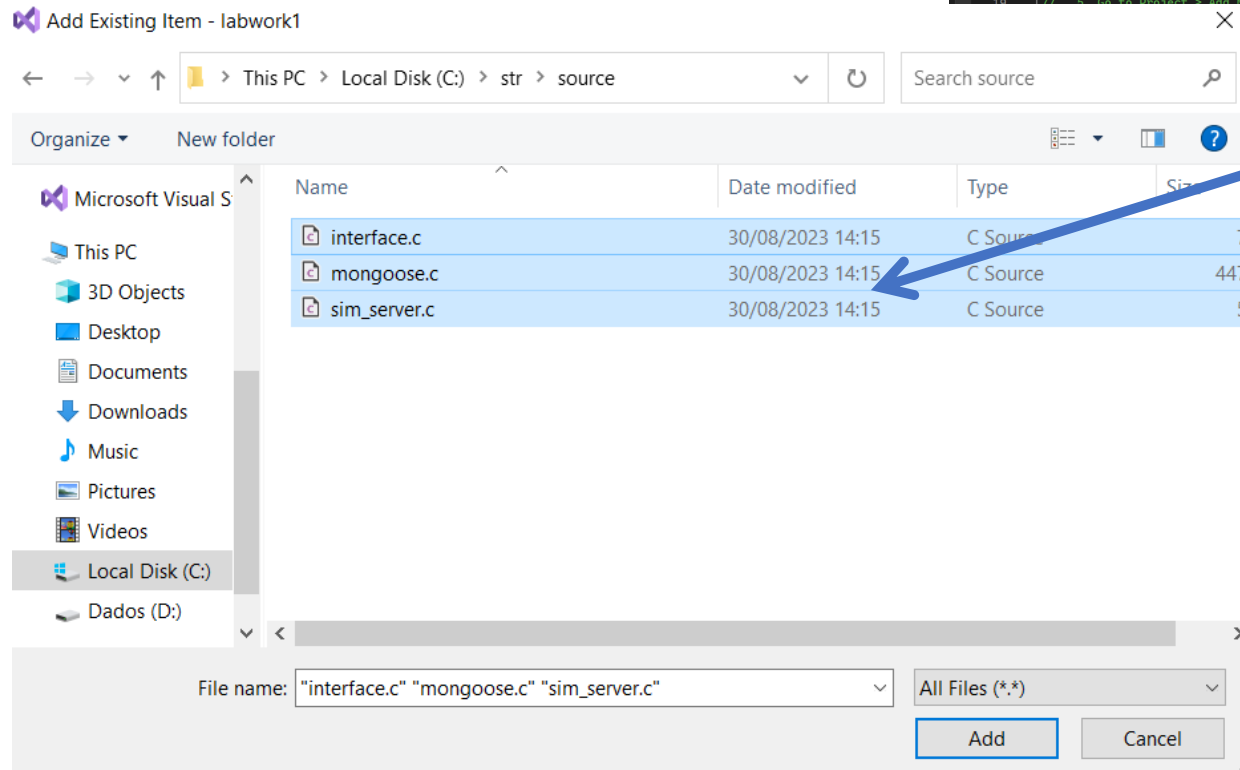
_CRT_SECURE_NO_WARNINGS;



LABWORK1 PROJECT



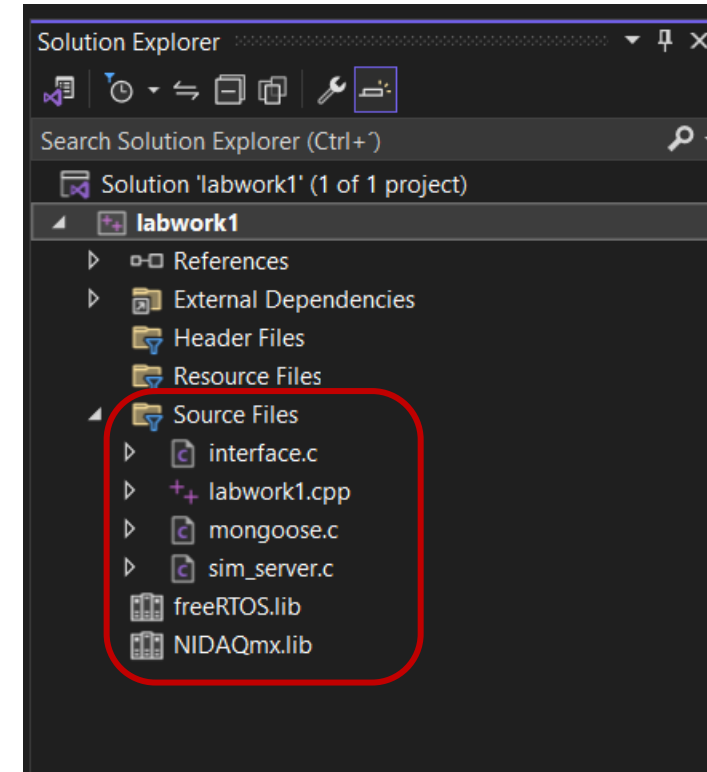
- Add source code to your project
 - Right click on source Files > Add > Existing Items



LABWORK1 PROJECT



- Aspect of your solution explorer in VS22
 - **sim_server.c** and **mongoose.c** handles the interaction with the kit simulator
 - **NIDAQmx.lib** allows your code to connect with the DAQ board.
 - **interface.c** allows switching between simulator and physical kit.
 - **FreeRTOS.lib** holds the realTime functionality for your code (next class)



LABWORK1 PROJECT: Example #1



❑ Try this example:

```
#include <iostream>
#include<conio.h>
#include<stdlib.h>
#include<windows.h> // for the Sleep function
```

```
extern "C" {
    #include <interface.h>
}
```

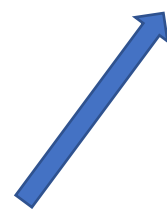
```
int main(int argc, char** argv) {
    createDigitalInput(0);
    createDigitalInput(1);
    createDigitalOutput(2);

    writeDigitalU8(2, 0x04); //moving conveyor
    Sleep(5000); //wait 5 seconds
    writeDigitalU8(2, 0); //stop all movements
    Sleep(5000); //wait 5 seconds
    closeChannels();
}
```

Copy this code inside your source file (probably the labwork1.cpp)

It's important!

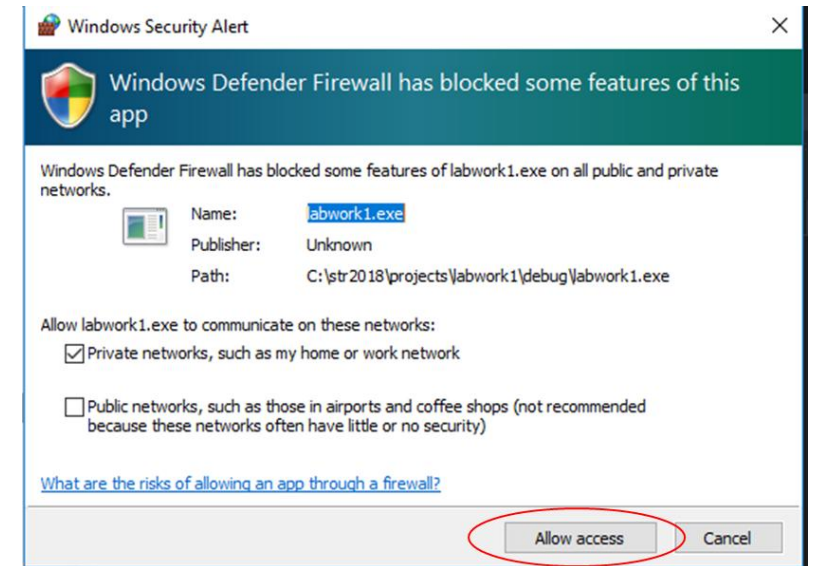
Allows using C and C++ together



❑ Run

❑ Allow the connection

❑ It lets your program create a connection with the simulator



LABWORK1 PROJECT: the simulator



❑ In a browser (e.g. Chrome) try: <http://localhost:8081/SplitterConveyorSim.html>

Case page not found:

- It means that simulator server is working, but page SplitterConveyorSim.html is not found.
- Probably your libraries for STR were copied into another folder, not the recommended one (c:\str\simulators)
- To solve this issue, either:
 - Copy your libraries to the recommend folder or
 - Modify macro MG_HTTP_FOLDER inside interface.c to the value corresponding to the location of your own simulators folder
 - Case there is a port conflict, that is, port 8081 being already used, change to other value.

```
// interface.c
// J. Rosas

#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <signal.h>
#include <winsock.h>
#include <interface.h>

// #define _USE_REALKIT_
```

```
#define MG_HTTP_FOLDER "c:/str/simulators"
#define MG_HTTP_PORT   "8081"
```

Not recommended!

LABWORK1 PROJECT: Run project

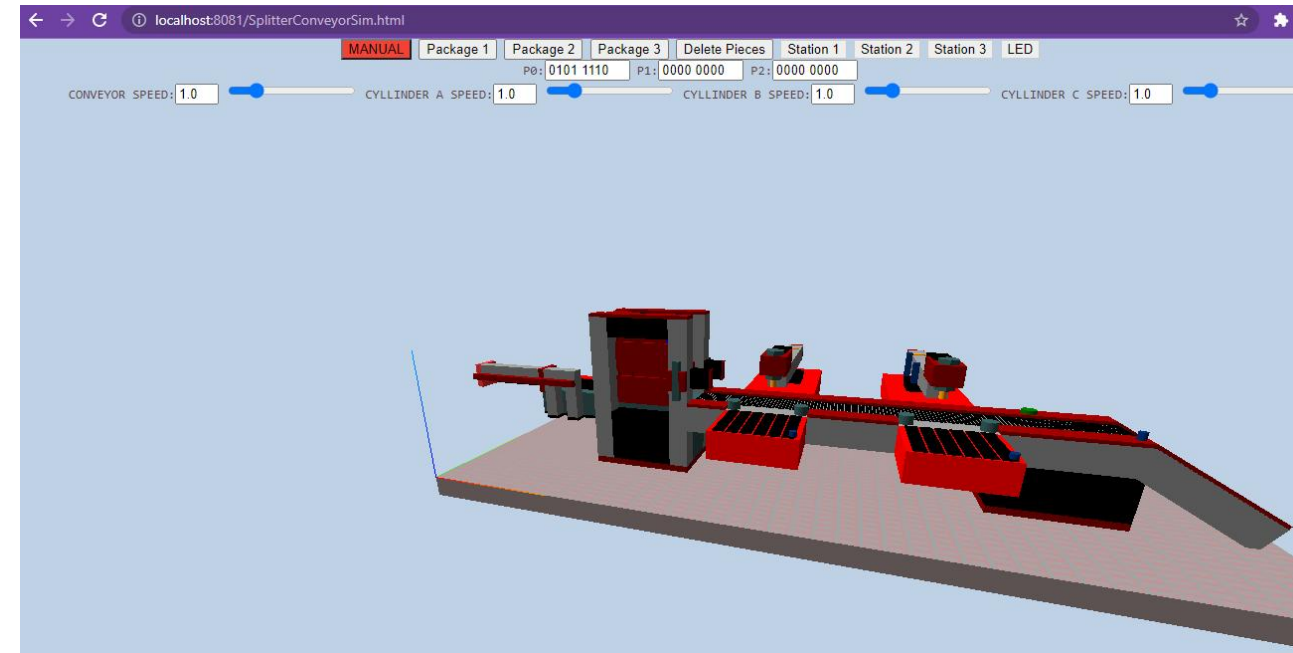


□ Run the labwork1 project

- Press F5 to run your project
- After it started running, copy the link <http://localhost:8081/SplitterConveyorSim.html> into your browser bar address, to start the simulator

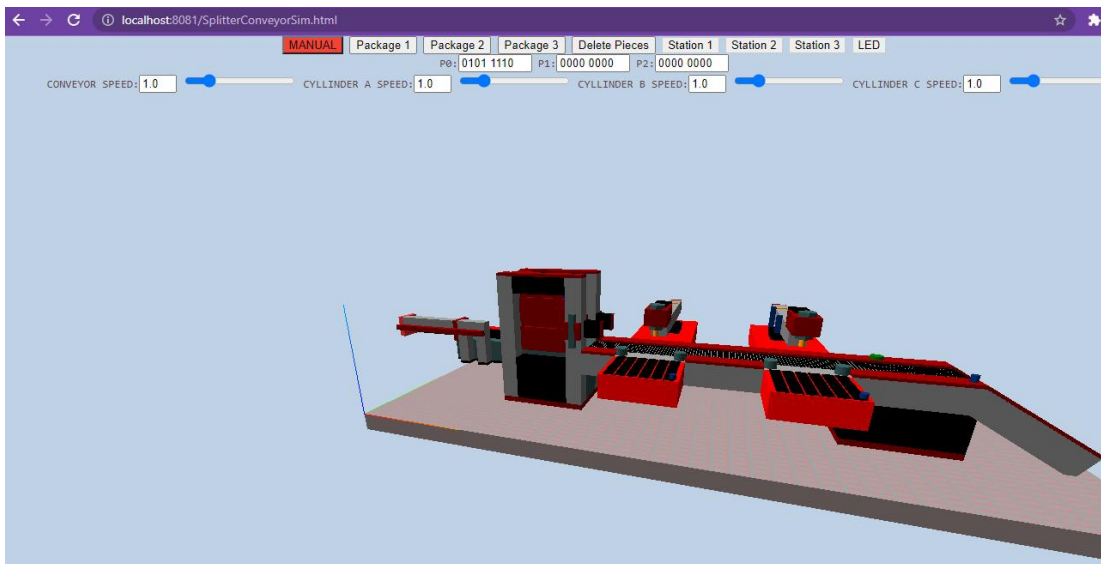
C:\str\labwork1\Debug\labwork1.exe

Starting web server on port 8081



LABWORK1 PROJECT: Example #2

- ❑ Try another example
- ❑ Use the code on the right into your source code file and try it... what's happening? How do we model the simulator movements?



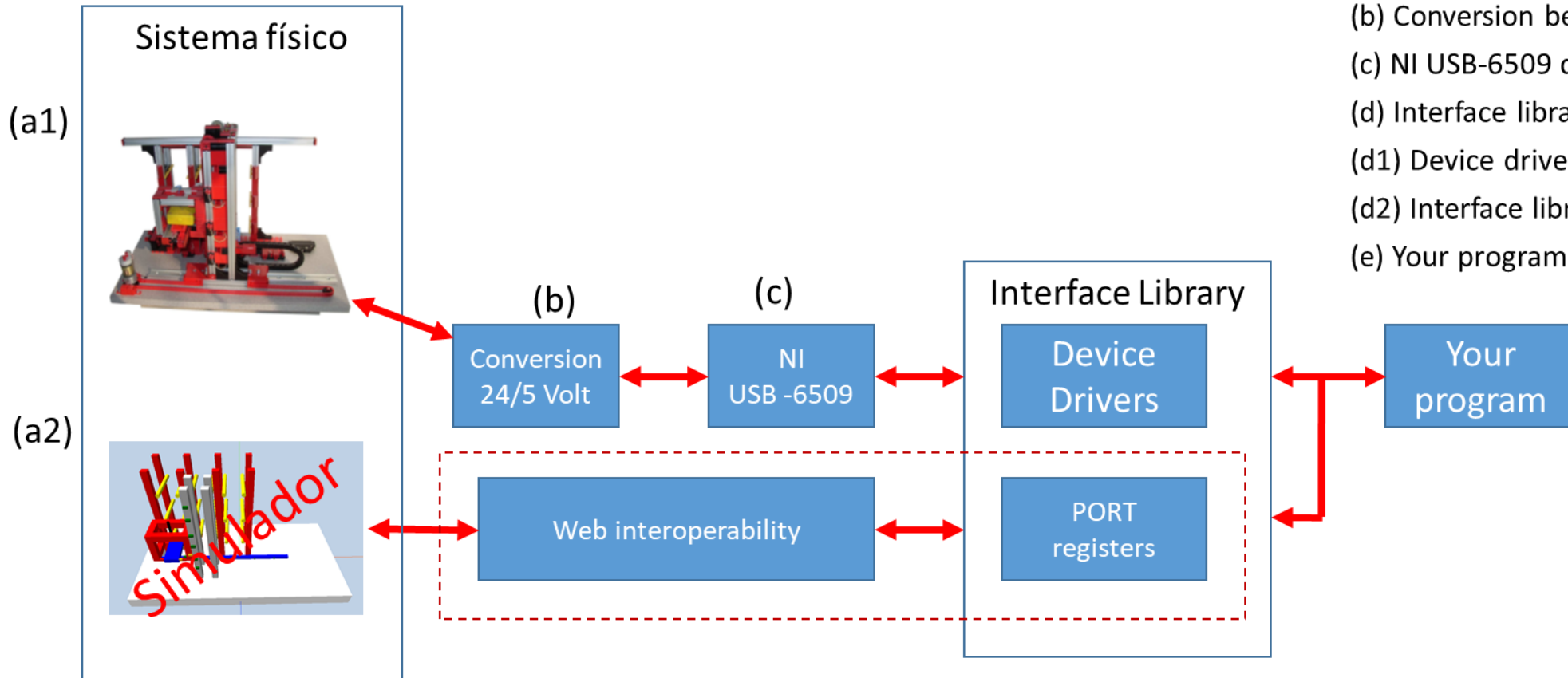
```
#include<conio.h>
#include<stdlib.h>
#include <windows.h> //for Sleep function
extern "C" {
    #include <interface.h>
}

int main(int argc, char **argv) {
    createDigitalInput (0);
    createDigitalInput (1);
    createDigitalOutput(2);
    uInt8 p0 = 0;
    uInt8 p2 = 0;
    writeDigitalU8(2, 0x2); // start moving cylinder start front
    while (!_kbhit()) { //do until a key is pressed
        p0 = readDigitalU8(0);
        p2 = readDigitalU8(2);
        if ((p0 & (1 << 5))) { // reached front position?
            p2 &= ~(1 << 1);
            p2 |= (1 << 0);
            writeDigitalU8(2, p2); // move cylinder start back
        }
        if ((p0 & (1 << 6))) { // reached back position?
            p2 &= ~(1 << 0);
            p2 |= (1 << 1);
            writeDigitalU8(2, p2); //move cylinder start front
        }
    }
    writeDigitalU8(2, 0);
    Sleep(5000);
    closeChannels();
}
```

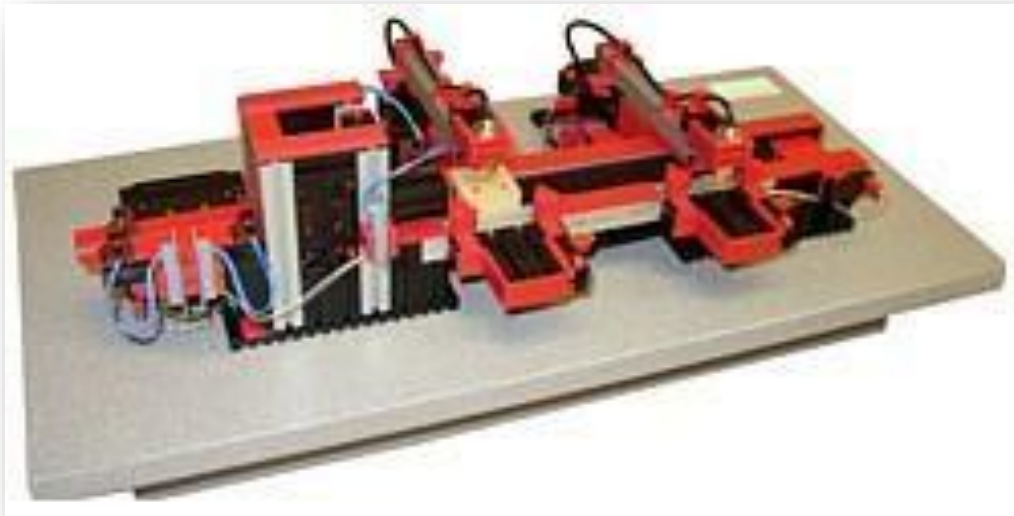
LABWORK1 PROJECT: Architecture Overview

- Switching between simulator/versus physical storage kit

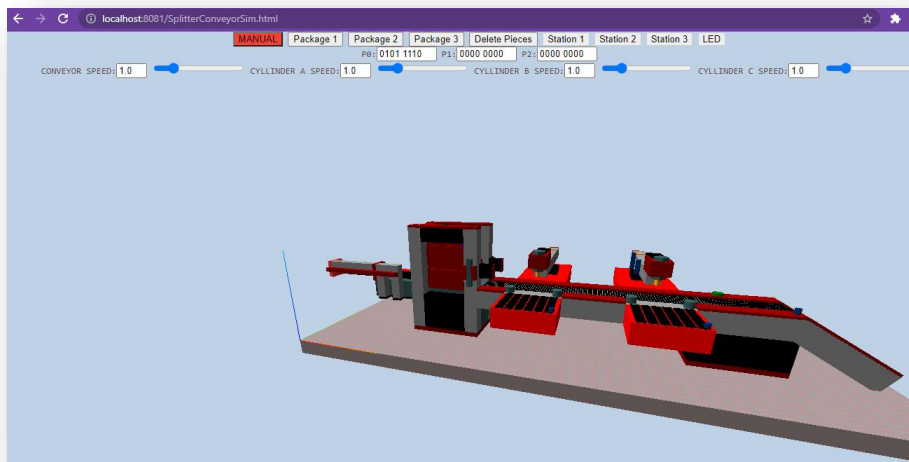
- (a1) Real system
- (a2) Corresponding simulator
- (b) Conversion between 24/5 Volt
- (c) NI USB-6509 digital acquisition board
- (d) Interface library (supplied)
 - (d1) Device drivers
 - (d2) Interface library
- (e) Your program



LABWORK1 PROJECT: Switching between kit & simulator



```
sim_server.c  interface.c  labwork1.cpp
labwork1 (Global Scope)
1  // interface.c
2  // J. Rosas
3
4  #include <stdlib.h>
5  #include <stdio.h>
6  #include <conio.h>
7  #include <signal.h>
8  #include <winsock.h>
9  #include <interface.h>
10
11  // #define _USE_REAL_KIT_
12
13  #define MG_HTTP_FOLDER "c:/str/simulators"
14  #define MG_HTTP_PORT  "8081"
15
```



LABWORK1 PROJECT: Work Assignment



☐ **Filling in the table below** (sensors / input – port 0 and port 1):

☐ CylinderStart (Back, Front)

☐ Conveyor (Left, Right)

☐ Cylinder1 (Back, Front, BrickDetection)

☐ Buttons (Dock1, Dock2 and DockEnd)

☐ Cylinder2 (Back, Front, BrickDetection)

Axis	Position	Tuple p<port.bit>	OBS.
CylinderStart	Back	P0.6	active high
	Front	P0.5	active high
Cylinder1	Back	P0.4	active low
	Front	P0.3	active low
Cylinder2	...	...	
	...	...	

LABWORK1 PROJECT: Work Assignment



☐ And the actuators (output – port 2) ??

Axis	Position	Tuple p<port.bit>	OBS.
CylinderStart	Back	P0.6	active high
	Front	P0.5	active high
Cylinder1	Back	P0.4	active low
	Front	P0.3	active low
Cylinder2	...	...	
	...	...	
CylinderStart	Move Front	P2.1	Active high
	Move Back	P2.0	Active high
	...	...	
	...	...	



☐ Move CylinderStart (Front, Back)

☐ Move Cylinder1 (Front, Back)

☐ Move Cylinder2 (Front, Back)

☐ Move Conveyor (move/stop)

☐ Led (on/off)

☐ **Complete the table!**

AWESOME! YOU'RE ALL SET TO START PLAYING WITH YOUR LAB 1 SYSTEM IN REAL-TIME SYSTEMS!



RELAX, EXPLORE, HAVE FUN, AND ENJOY THE RIDE!



KEEP UP THE
GOOD
WORK!